HLD Level 1.2

Networking & Communication - C++ / Systems Interview Notes

Goal: when two components in an architecture need to communicate, choose an appropriate communication model and explain the trade-offs. For C++ systems roles, be ready to go below the API level into sockets, streams, connection handling, buffers and I/O models.

1. What you need to decide

Topic	Decision
TCP vs UDP	Reliable ordered byte stream vs datagram-oriented transport
HTTP / REST	Simple request-response APIs, especially external/public interfaces
gRPC	Typed RPC, efficient internal service-to-service communication
WebSocket	Persistent bidirectional client-server communication
Polling / Long Polling	Periodic requests vs waiting requests
Streaming	Continuous flow of updates/data
Sync vs Async	Caller waits for result vs work is decoupled
Persistent connections	Reuse connection vs repeated connect/disconnect
Serialization	Human-readable vs compact binary representation

2. TCP

TCP provides a **reliable, ordered, connection-oriented byte stream**. Lost data is retransmitted and bytes are delivered to the application in order. It is a natural choice when losing commands, transactions or configuration updates is unacceptable.

Important C++ interview point: TCP preserves bytes, not application message boundaries.

```
Sender:
send("HELLO", 5)
send("WORLD", 5)

Receiver might observe:
recv -> "HEL"
recv -> "LOWORLD"

or one recv -> "HELLOWORLD"
```

Therefore the application protocol needs framing. Common approaches include a fixed-size message, delimiter-based framing, or a length-prefixed message such as [length][payload]. Also remember that send/write may write fewer bytes than requested and recv/read may return fewer bytes than the complete logical message.

3. UDP

UDP is **connectionless and datagram-oriented**. It does not inherently guarantee delivery, ordering, retransmission or duplicate prevention. The application receives datagrams rather than a TCP-style byte stream.

UDP becomes attractive when timeliness matters more than perfect delivery, or when the application/protocol wants direct control over reliability behavior. Examples can include real-time media, games, discovery protocols and specialized low-latency systems.

Decision question: Is losing some old information worse than delaying newer information?

4. TCP vs UDP - interview reasoning

Critical device command: TCP is usually easier to justify because reliable ordered delivery matters.

Real-time media: retransmitting old data may increase latency; depending on requirements, a UDP-based protocol may prefer continuing with fresh data.

Do not say: UDP is always faster or TCP is always better. State the reliability/latency requirement first.

5. HTTP and REST

HTTP is an application protocol commonly used for request-response communication. REST is an API design style commonly implemented over HTTP.

```
REST / HTTP / TCP / IP
```

So 'TCP vs REST' is not a clean comparison. A REST API using HTTP/1.1 or HTTP/2 commonly runs over TCP. In an interview, separate the application/API choice from the underlying transport.

Typical REST operations

```
GET /devices/123
POST /orders
PUT /devices/123
DELETE /devices/123
```

REST/HTTP is attractive for interoperability, public APIs, browser/tool support and operational simplicity. It is often easy to inspect and debug.

6. gRPC

gRPC provides RPC-style service interfaces and commonly uses Protocol Buffers for typed message definitions and efficient binary serialization. It supports unary and streaming RPC patterns and is often attractive for internal service-to-service communication.

```
Example conceptual interface:

TelemetryService
SendTelemetry(...)
GetDevice(...)
```

Choose based on interface contracts, performance, streaming needs, interoperability, tooling and ecosystem. 'gRPC is faster' is not sufficient justification by itself.

7. WebSocket

WebSocket provides a long-lived bidirectional channel between client and server. Once established, either side can send messages without creating a fresh request for every update.

```
Browser <--------------------> Server
persistent connection

Server -> telemetry update
Server -> alert
Server -> device offline
```

Useful for chat, live dashboards, frequent notifications and collaborative applications. The cost is that the server now manages persistent connection state and associated resources.

8. Polling, long polling and streaming

Polling: client periodically asks whether anything changed. Simple, but frequent empty requests can waste resources and updates arrive only at polling intervals.

Long polling: server holds a request until data arrives or a timeout occurs; the client then issues another request.

Streaming: data continuously flows over a longer-lived communication channel. Useful when updates are frequent.

9. Synchronous vs asynchronous communication

Synchronous: A calls B and waits because B's result is required before A can continue.

Asynchronous: A submits work/event and continues; processing happens independently.

```
A -> B -> response -> A continues

versus

A -> Queue -> B
A continues independently
```

Async communication can improve decoupling, burst handling, independent scaling and failure isolation, but introduces queueing, retries, duplicates, ordering concerns and eventual processing.

10. Persistent vs short-lived connections

Repeatedly connecting and disconnecting for frequent communication adds connection-management overhead. Persistent connections can reduce this overhead and improve latency, but they require the server to manage connection state, timeouts, heartbeats, reconnection and per-connection resources.

```
Short-lived:
connect -> send -> disconnect -> connect -> send -> disconnect

Persistent:
connect -> send -> send -> send -> ... -> disconnect
```

For large device fleets, always ask how many concurrent connections the service must support and what memory/file-descriptor cost each connection introduces.

11. Connection lifecycle and failure handling

A production connection is more than connect/send/receive. Think through:

```
resolve/connect -> authenticate/handshake -> communicate -> idle/heartbeat -> detect
timeout/error -> close -> backoff -> reconnect
```

Important topics: connect timeout, read/write timeout, heartbeat, half-open/stale connection detection, retry limits, exponential backoff, jitter, graceful close and reconnect behavior.

12. Blocking vs non-blocking I/O

Blocking I/O: a thread can wait while an operation completes. This is simple and may be completely reasonable at moderate connection counts.

Non-blocking/event-driven I/O: the program manages many sockets without dedicating a blocked thread to every connection.

```
Thread-per-connection:
Connection 1 -> Thread 1
Connection 2 -> Thread 2
Connection 3 -> Thread 3

Event-driven:
many sockets -> event loop -> ready events -> handlers/workers
```

On Linux, **epoll** is an important mechanism for efficiently waiting for readiness events across many file descriptors. You do not need to implement an epoll server for basic HLD, but for C++/Linux interviews you should understand why event-driven I/O becomes attractive at very high connection counts.

13. Thread-per-connection vs event-driven

Do not blindly claim one model is always superior. Consider connection count, how active connections are, complexity, blocking work, CPU cost, latency targets and available resources.

10 active connections -> thread-per-connection may be simple and adequate.
100,000 mostly idle persistent connections -> event-driven I/O is usually far more attractive than 100,000 dedicated threads.

14. Serialization

Components need a common representation for messages. JSON is human-readable and broadly interoperable, but typically larger and more expensive to parse than compact binary formats. Protocol Buffers provide a schema, typed generated code and compact binary serialization, which can be attractive for internal C++ services.

The decision is not only speed: consider compatibility, schema evolution, debuggability, payload size and language ecosystem.

15. C++ networking details interviewers may probe

Be ready to explain these concepts at a high level:

- socket/file descriptor
- partial send/write and partial recv/read
- message framing over TCP
- buffers and ownership
- blocking vs non-blocking sockets
- select/poll/epoll conceptually
- worker/thread pools
- timeouts and cancellation
- connection limits and file descriptors
- heartbeat/reconnect behavior
- graceful shutdown

```
A strong C++ HLD candidate can zoom from:

Devices -> Gateway -> Processing Service

down into:

Gateway -> event loop -> socket buffers -> bounded work queue -> worker pool
```

16. Communication decision tree

When two components need to communicate, ask:

1. Must delivery be reliable?
2. Does ordering matter?
3. Can duplicates occur/be tolerated?
4. Is it request-response or continuous communication?
5. Does the caller need the result immediately?
6. How frequent are messages?
7. How many concurrent connections?
8. What latency is acceptable?
9. Can stale/lost messages be tolerated?
10. What happens when the peer/network fails?

17. Common interview mistakes

TCP preserves messages: false - TCP is a byte stream.

One send equals one recv: false.

UDP means unreliable application: too simplistic - applications/protocols can add reliability mechanisms.

REST vs TCP: mixes different layers.

WebSocket for everything real-time: requirements and clients matter.

100k connections = 100k threads: usually a warning sign.

Infinite reconnect retries: can create retry storms.

Ignoring failure: every network call can timeout, fail or return late.

18. Six interview scenarios

1. A device receives critical configuration/reboot commands. TCP or UDP? Why?

2. You are transporting live video. Would you always choose TCP? What trade-off determines the answer?

3. A browser dashboard needs frequent live telemetry. Polling or WebSocket? Explain.

4. Two internal backend services communicate. REST vs gRPC - what factors decide?

5. A Linux C++ gateway must maintain 100,000 mostly idle connections. Thread-per-connection or event-driven? Why?

6. A sender performs two TCP send() calls. Can the receiver assume two recv() calls return exactly those two messages? Explain how you would frame messages.

19. 60-second revision sheet

TCP: reliable, ordered byte stream; no message boundaries.
UDP: datagrams; no inherent delivery/order guarantee.
REST: API style commonly over HTTP.
gRPC: typed RPC, Protobuf, streaming support.
WebSocket: persistent bidirectional channel.
Polling: repeated request for updates.
Async: decouples caller from processing.
Persistent connection: lower repeated setup cost, higher state/resource responsibility.
High connection count: consider event-driven/non-blocking I/O.
Always design: timeout + failure detection + reconnect/backoff.

20. Ready for Level 1.3 when...

You can choose and defend a communication mechanism from requirements, explain TCP stream semantics, distinguish REST/gRPC/WebSocket, reason about persistent connections, and explain why an event-driven Linux C++ server may be preferred at high connection counts.